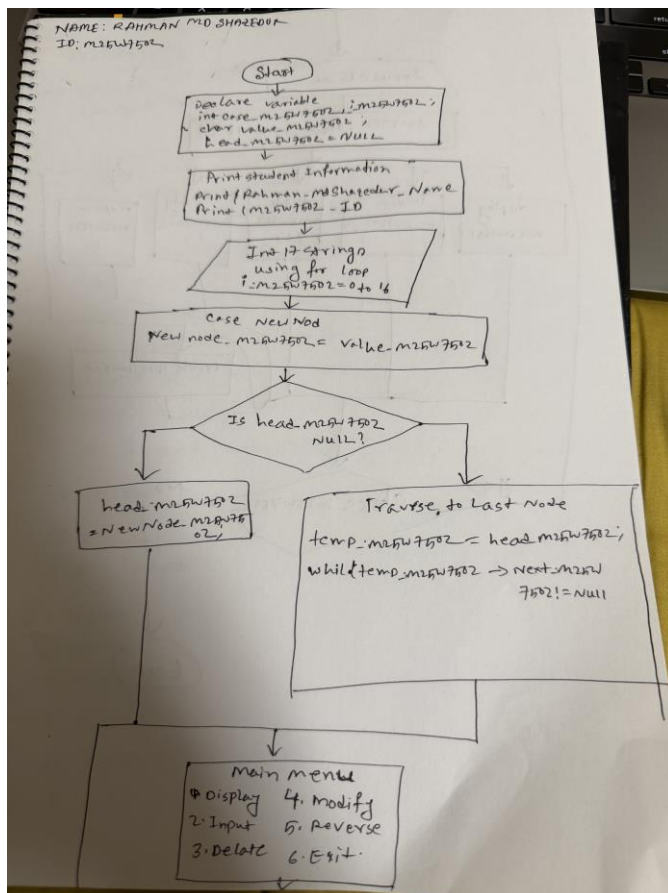


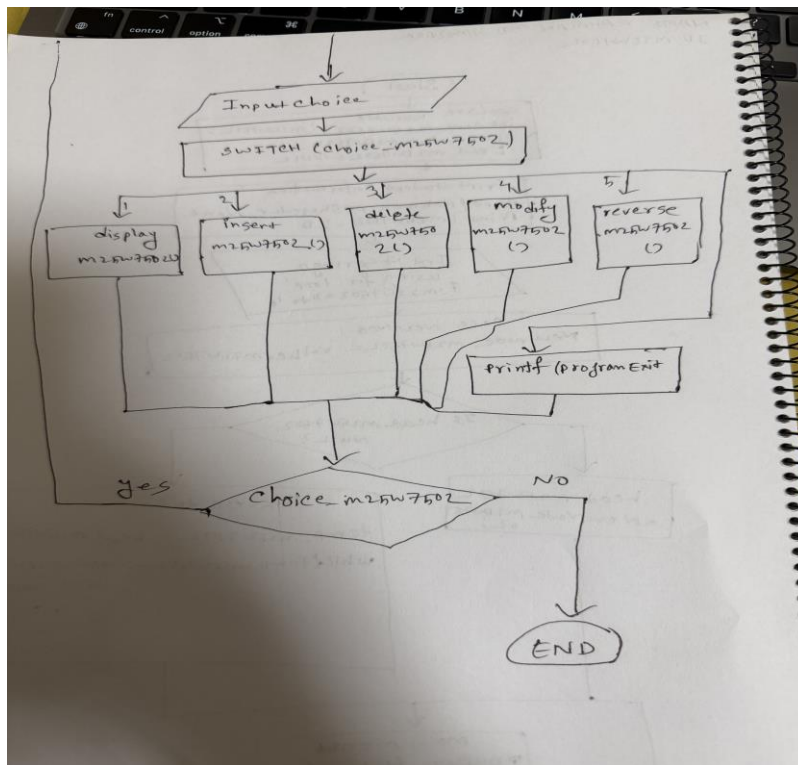
Algorithms Assignment 04

Full Name: RAHMAN MD SHAZEDUER

ID: M25W7502

1. Flowcharts





Pseudocode

Pseudo Code:

```

Name: RAHMAN MD SHAZEDUR
ID: m15w7502

Start Node - m15w7502
Declare head - m15w7502
Display student Name And ID
Name: RAHMAN MD SHAZEDUR
ID: m15w7502
Input 17 country Name
For i: m15w7502 = 1 to 17
Create newNode - m15w7502
Store country name - m15w7502
Insert node at END
END For
DO
Display menu
1. Insert
2. Delete
3. modify
4. Display
5. Reverse
6. Exit

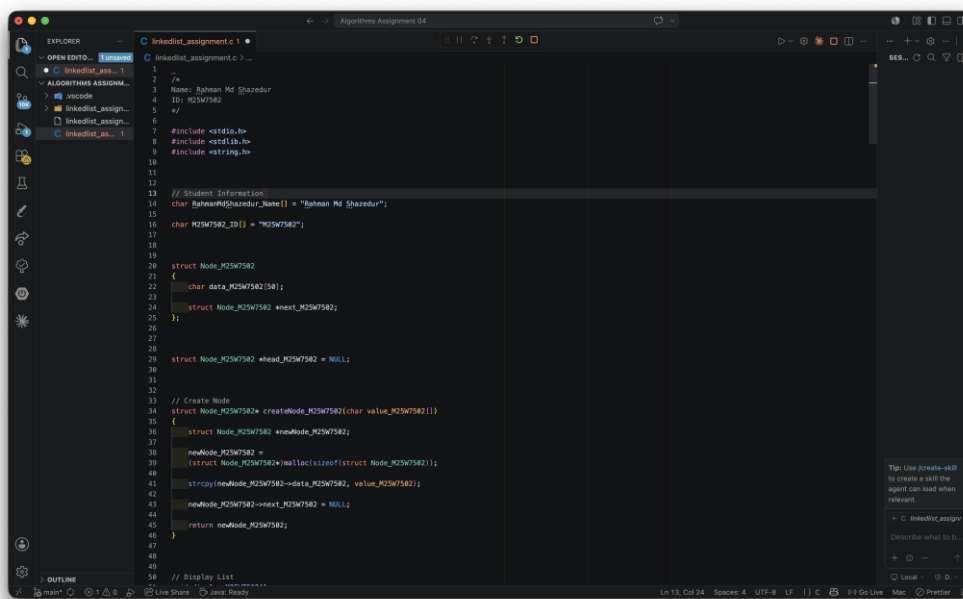
```

```

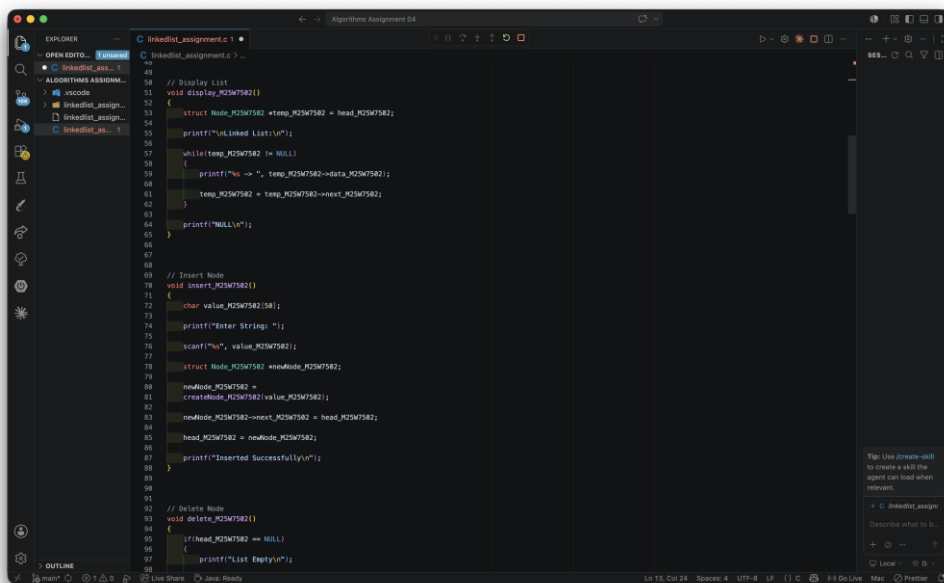
Input menu choice - m15w7502
IF choice == 1
    Call insert - m15w7502 ()
Else IF choice == 2
    Call delete - m15w7502 ()
Else IF choice == 3
    Call modify - m15w7502 ()
Else IF choice == 4
    Call display - m15w7502 ()
Else IF choice == 5
    Call reverse - m15w7502 ()
Else IF choice == 6
    Stop
Else
    Display Invalid choice
END IF
WHILE menu choice - m15w7502 != 6
END

```

Code



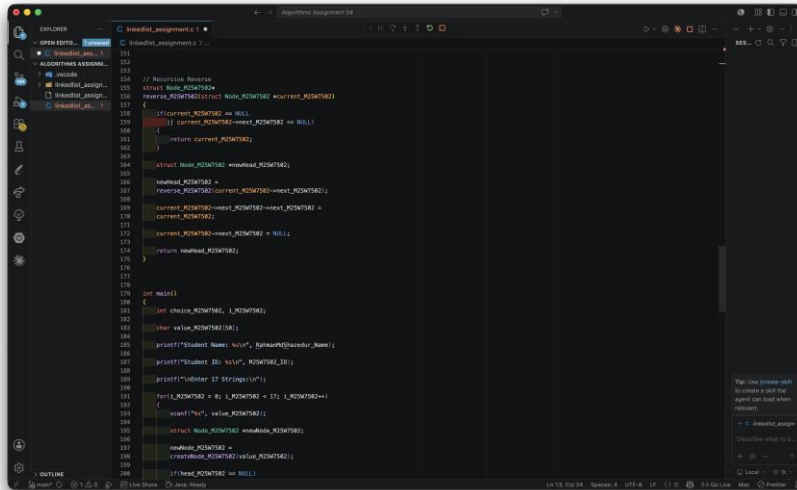
```
1 // linkedlist_assignment.c
2 /*
3  * Name: Rahman Mj Shazedur
4  * ID: M25W7582
5  */
6
7 #include <stdio.h>
8 #include <stdlib.h>
9 #include <string.h>
10
11
12 // Student Information
13
14 char RahmanMjShazedur_Name[] = "Rahman Mj Shazedur";
15 char M25W7582_ID[] = "M25W7582";
16
17
18
19
20 struct Node_M25W7582
21 {
22     char data_M25W7582[50];
23     struct Node_M25W7582 *next_M25W7582;
24 };
25
26
27
28 struct Node_M25W7582 *head_M25W7582 = NULL;
29
30
31
32 // Create Node
33
34 struct Node_M25W7582 *createNode_M25W7582(char value_M25W7582[])
35 {
36     struct Node_M25W7582 *newNode_M25W7582;
37
38     newNode_M25W7582 =
39         (struct Node_M25W7582 *) malloc(sizeof(struct Node_M25W7582));
40
41     strcpy(newNode_M25W7582->data_M25W7582, value_M25W7582);
42     newNode_M25W7582->next_M25W7582 = NULL;
43
44     return newNode_M25W7582;
45 }
46
47
48
49 // Display List
```



```
50 // Display List
51 void display_M25W7582()
52 {
53     struct Node_M25W7582 *temp_M25W7582 = head_M25W7582;
54
55     printf("\nLinked List:\n");
56
57     while(temp_M25W7582 != NULL)
58     {
59         printf("%s -> ", temp_M25W7582->data_M25W7582);
60
61         temp_M25W7582 = temp_M25W7582->next_M25W7582;
62     }
63
64     printf("\n");
65
66 }
67
68
69 // Insert Node
70 void insert_M25W7582()
71 {
72     char value_M25W7582[50];
73
74     printf("Enter String: ");
75
76     scanf("%s", value_M25W7582);
77
78     struct Node_M25W7582 *newNode_M25W7582;
79
80     newNode_M25W7582 =
81         createNode_M25W7582(value_M25W7582);
82
83     newNode_M25W7582->next_M25W7582 = head_M25W7582;
84
85     head_M25W7582 = newNode_M25W7582;
86
87     printf("Inserted Successfully!\n");
88 }
89
90
91 // Delete Node
92 void delete_M25W7582()
93 {
94     if(head_M25W7582 == NULL)
95     {
96         printf("List Empty!\n");
97     }
98 }
```

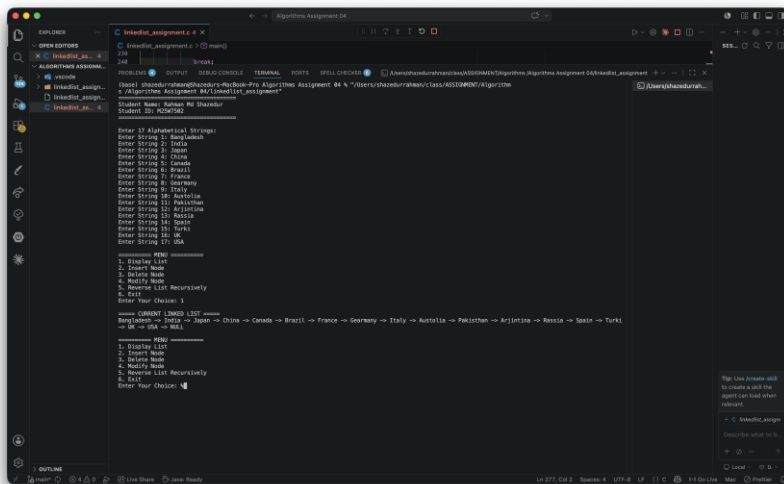
```
101 void delete_RCNW7582()
102 {
103     if(head_RCNW7582 == NULL)
104     {
105         printf("List Empty\n");
106         return;
107     }
108     struct Node_RCNW7582 *temp_RCNW7582 = head_RCNW7582;
109     head_RCNW7582 = head_RCNW7582->next_RCNW7582;
110     free(temp_RCNW7582);
111     printf("Deleted Successfully\n");
112 }
113
114 // Modify Node
115 void modify_RCNW7582()
116 {
117     int position_RCNW7582, i_RCNW7582;
118     struct Node_RCNW7582 *temp_RCNW7582 = head_RCNW7582;
119     printf("Enter Position: ");
120     scanf("%d", &position_RCNW7582);
121     printf("Enter New String: ");
122     scanf("%s", &temp_RCNW7582);
123     if(temp_RCNW7582 == NULL)
124     {
125         printf("Invalid Position\n");
126         return;
127     }
128     if(position_RCNW7582 == 1)
129     {
130         temp_RCNW7582->data_RCNW7582 = temp_RCNW7582;
131         temp_RCNW7582->next_RCNW7582 = temp_RCNW7582->next_RCNW7582;
132         free(temp_RCNW7582);
133     }
134     else
135     {
136         temp_RCNW7582->data_RCNW7582 = temp_RCNW7582;
137         temp_RCNW7582->next_RCNW7582 = temp_RCNW7582->next_RCNW7582;
138         free(temp_RCNW7582);
139     }
140     printf("Modified Successfully\n");
141 }
```

```
142 // Insert Node
143 void insert_RCNW7582()
144 {
145     struct Node_RCNW7582 *temp_RCNW7582 = head_RCNW7582;
146     if(head_RCNW7582 == NULL)
147     {
148         temp_RCNW7582 = (struct Node_RCNW7582 *)malloc(sizeof(struct Node_RCNW7582));
149         temp_RCNW7582->data_RCNW7582 = temp_RCNW7582;
150         temp_RCNW7582->next_RCNW7582 = temp_RCNW7582;
151         head_RCNW7582 = temp_RCNW7582;
152     }
153     else
154     {
155         printf("List Display\n");
156         printf("Enter: ");
157         scanf("%s", &temp_RCNW7582);
158         printf("Enter Position: ");
159         scanf("%d", &position_RCNW7582);
160         if(position_RCNW7582 == 1)
161         {
162             temp_RCNW7582->data_RCNW7582 = temp_RCNW7582;
163             temp_RCNW7582->next_RCNW7582 = temp_RCNW7582->next_RCNW7582;
164             free(temp_RCNW7582);
165         }
166         else
167         {
168             temp_RCNW7582->data_RCNW7582 = temp_RCNW7582;
169             temp_RCNW7582->next_RCNW7582 = temp_RCNW7582->next_RCNW7582;
170             free(temp_RCNW7582);
171         }
172     }
173     printf("Inserted Successfully\n");
174 }
```

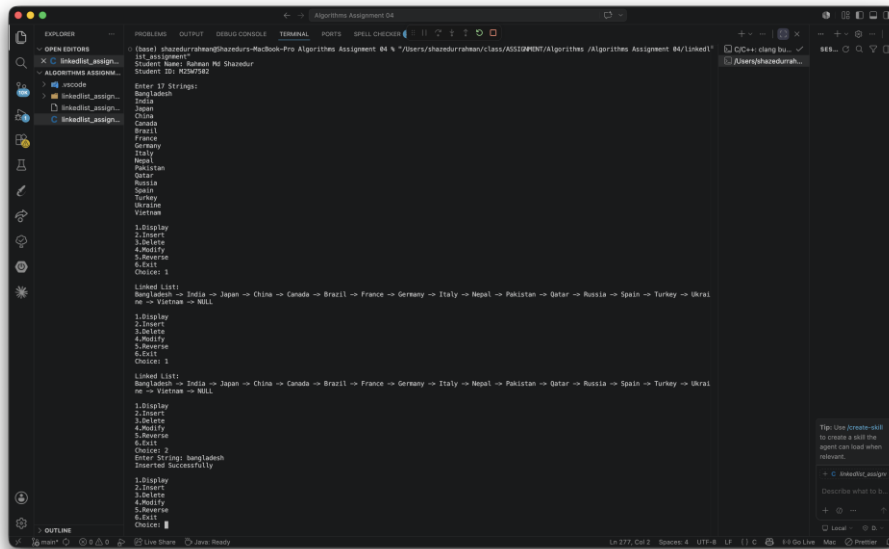


OUTPUT

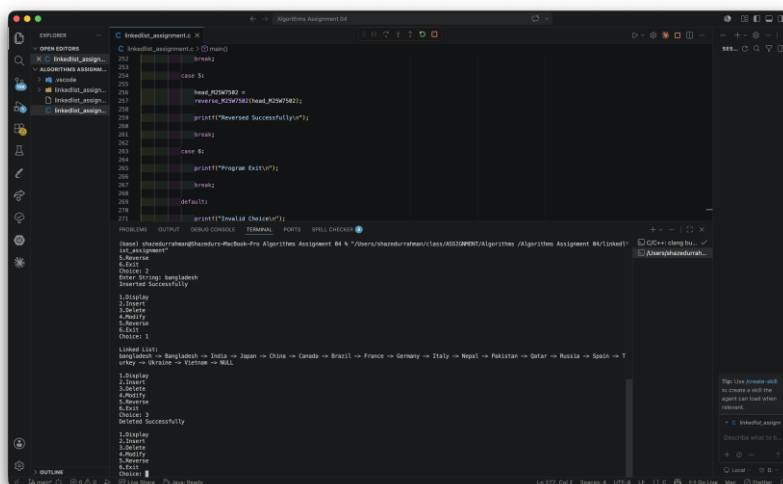
1. Output



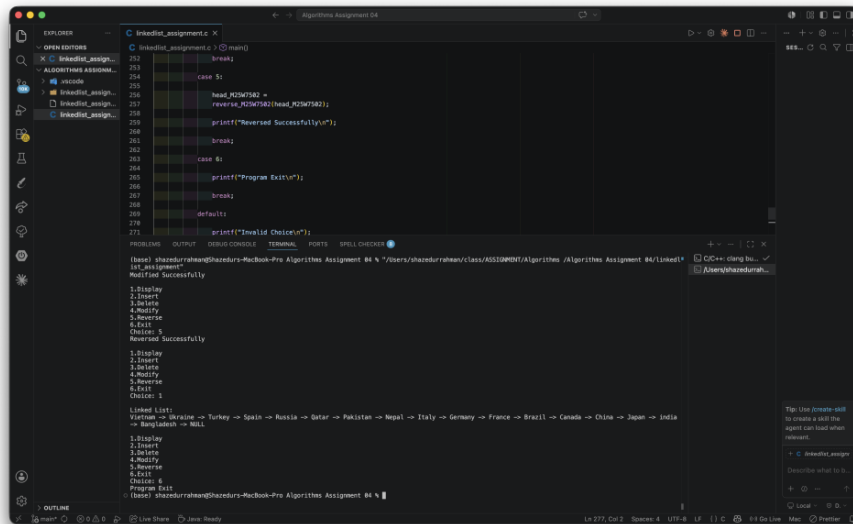
2 — Insert Operation



3 — Delete Operation



4 — Modify Operation



Code.....

/*

Name: Rahman Md Shazedur

ID: M25W7502

*/

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
// Student Information
```

```
char RahmanMdShazedur_Name[] = "Rahman Md Shazedur";
```

```
char M25W7502_ID[] = "M25W7502";
```

```
struct Node_M25W7502
```

```
{
```

```
char data_M25W7502[50];
```

```
struct Node_M25W7502 *next_M25W7502;
```

```
};
```

```
struct Node_M25W7502 *head_M25W7502 = NULL;
```

```
// Create Node
```

```
struct Node_M25W7502* createNode_M25W7502(char value_M25W7502[])
```

```
{  
    struct Node_M25W7502 *newNode_M25W7502;  
  
    newNode_M25W7502 =  
    (struct Node_M25W7502*)malloc(sizeof(struct Node_M25W7502));  
  
    strcpy(newNode_M25W7502->data_M25W7502, value_M25W7502);  
  
    newNode_M25W7502->next_M25W7502 = NULL;  
  
    return newNode_M25W7502;  
}
```

// Display List

```
void display_M25W7502()  
{  
    struct Node_M25W7502 *temp_M25W7502 = head_M25W7502;  
  
    printf("\nLinked List:\n");  
  
    while(temp_M25W7502 != NULL)  
    {  
        printf("%s -> ", temp_M25W7502->data_M25W7502);  
  
        temp_M25W7502 = temp_M25W7502->next_M25W7502;  
    }  
}
```

```
printf("NULL\n");
```

```
}
```

```
// Insert Node
```

```
void insert_M25W7502()
```

```
{
```

```
char value_M25W7502[50];
```

```
printf("Enter String: ");
```

```
scanf("%s", value_M25W7502);
```

```
struct Node_M25W7502 *newNode_M25W7502;
```

```
newNode_M25W7502 =
```

```
createNode_M25W7502(value_M25W7502);
```

```
newNode_M25W7502->next_M25W7502 = head_M25W7502;
```

```
head_M25W7502 = newNode_M25W7502;
```

```
printf("Inserted Successfully\n");
```

```
}
```

```
// Delete Node
```

```
void delete_M25W7502()
```

```
{
```

```
if(head_M25W7502 == NULL)
```

```
{
```

```
printf("List Empty\n");
```

```
return;
```

```
}
```

```
struct Node_M25W7502 *temp_M25W7502 = head_M25W7502;
```

```
head_M25W7502 = head_M25W7502->next_M25W7502;
```

```
free(temp_M25W7502);
```

```
printf("Deleted Successfully\n");
```

```
}
```

```
// Modify Node
```

```
void modify_M25W7502()
```

```
{
```

```
int position_M25W7502, i_M25W7502;
```

```
char value_M25W7502[50];
```

```
struct Node_M25W7502 *temp_M25W7502 = head_M25W7502;
```

```
printf("Enter Position: ");
```

```
scanf("%d", &position_M25W7502);
```

```
printf("Enter New String: ");
```

```
scanf("%s", value_M25W7502);
```

```
for(i_M25W7502 = 1;  
i_M25W7502 < position_M25W7502  
&& temp_M25W7502 != NULL;  
i_M25W7502++)  
{  
temp_M25W7502 =  
temp_M25W7502->next_M25W7502;  
}
```

```
if(temp_M25W7502 == NULL)  
{  
printf("Invalid Position\n");
```

```
return;  
}
```

```
strcpy(temp_M25W7502->data_M25W7502,
```

```
value_M25W7502);
```

```
printf("Modified Successfully\n");
```

```
}
```

```
// Recursive Reverse
```

```
struct Node_M25W7502*
```

```
reverse_M25W7502(struct Node_M25W7502 *current_M25W7502)
```

```
{
```

```
if(current_M25W7502 == NULL
```

```
|| current_M25W7502->next_M25W7502 == NULL)
```

```
{
```

```
return current_M25W7502;
```

```
}
```

```
struct Node_M25W7502 *newHead_M25W7502;
```

```
newHead_M25W7502 =
```

```
reverse_M25W7502(current_M25W7502->next_M25W7502);
```

```
current_M25W7502->next_M25W7502->next_M25W7502 =
```

```
current_M25W7502;
```

```
current_M25W7502->next_M25W7502 = NULL;
```

```
return newHead_M25W7502;
```

```
}
```

```
int main()
```

```
{
```

```
int choice_M25W7502, i_M25W7502;
```

```
char value_M25W7502[50];
```

```
printf("Student Name: %s\n", RahmanMdShazedur_Name);
```

```
printf("Student ID: %s\n", M25W7502_ID);
```

```
printf("\nEnter 17 Strings:\n");
```

```
for(i_M25W7502 = 0; i_M25W7502 < 17; i_M25W7502++)
```

```
{
```

```
scanf("%s", value_M25W7502);
```

```
struct Node_M25W7502 *newNode_M25W7502;
```

```
newNode_M25W7502 =
```

```
createNode_M25W7502(value_M25W7502);
```

```
if(head_M25W7502 == NULL)
```

```
{
```

```
head_M25W7502 = newNode_M25W7502;
```



```

}

else

{

struct Node_M25W7502 *temp_M25W7502 =

head_M25W7502;


while(temp_M25W7502->next_M25W7502 != NULL)

{

temp_M25W7502 =

temp_M25W7502->next_M25W7502;

}


temp_M25W7502->next_M25W7502 =

newNode_M25W7502;

}

}


do

{

printf("\n1.Display\n2.Insert\n3.Delete\n4.Modify\n5.Reverse\n6.Exit\n");


printf("Choice: ");


scanf("%d", &choice_M25W7502);


switch(choice_M25W7502)

{

case 1:

```

```
display_M25W7502();
```

```
break;
```

```
case 2:
```

```
insert_M25W7502();
```

```
break;
```

```
case 3:
```

```
delete_M25W7502();
```

```
break;
```

```
case 4:
```

```
modify_M25W7502();
```

```
break;
```

```
case 5:
```

```
head_M25W7502 =
```

```
reverse_M25W7502(head_M25W7502);
```

```
printf("Reversed Successfully\n");
```

```
break;
```

```
case 6:
```

```
printf("Program Exit\n");
```

```
break;
```

```
default:
```

```
printf("Invalid Choice\n");
```

```
}
```

```
} while(choice_M25W7502 != 6);
```

```
return 0;
```

```
}
```